ESP32 Pico and Thonny IDE

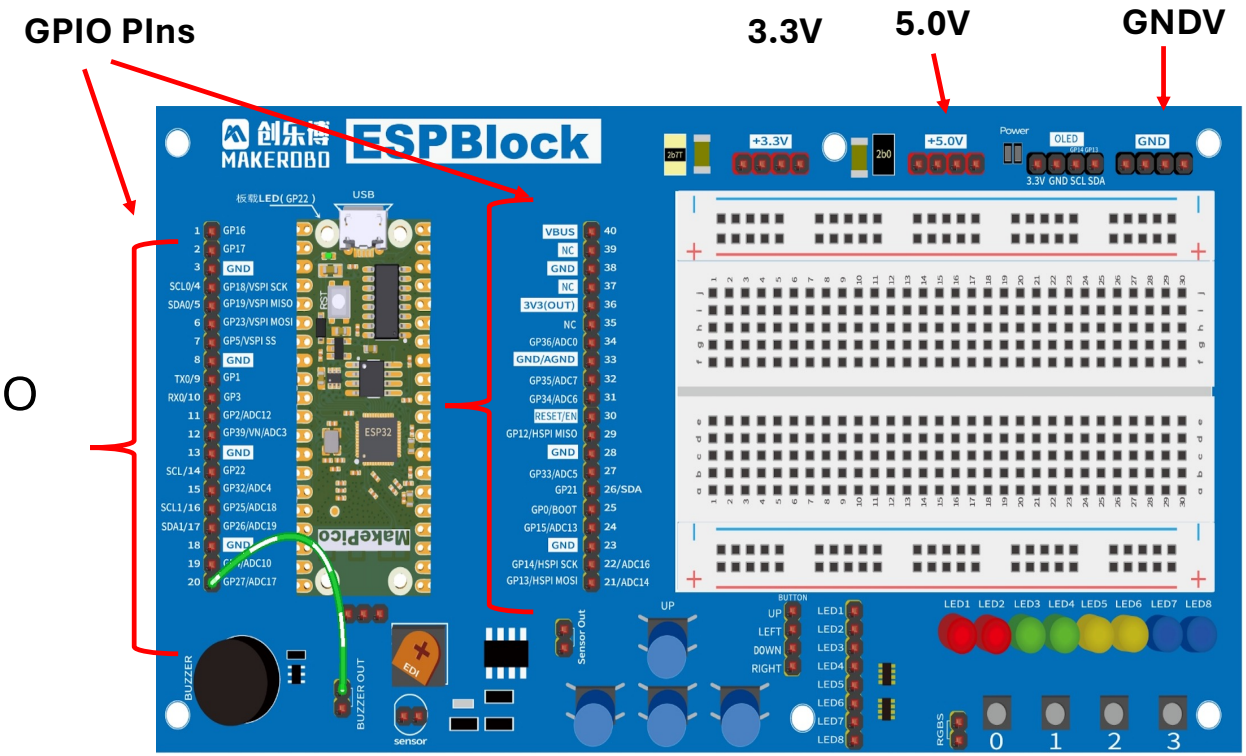
Lab Session 2

Outline

- ESPBlock-Expansion Board
- Sensor Experiments - continues

The ESPBlock- Expansion Board

ESP32 is a microcontroller with Flash memory. It has Built-in Wi-Fi and Bluetooth with 40 GPIO pins. Includes ADC (Analog input) PWM (Motor/LED control) UART, I2C, SPI communication



Mind the connections

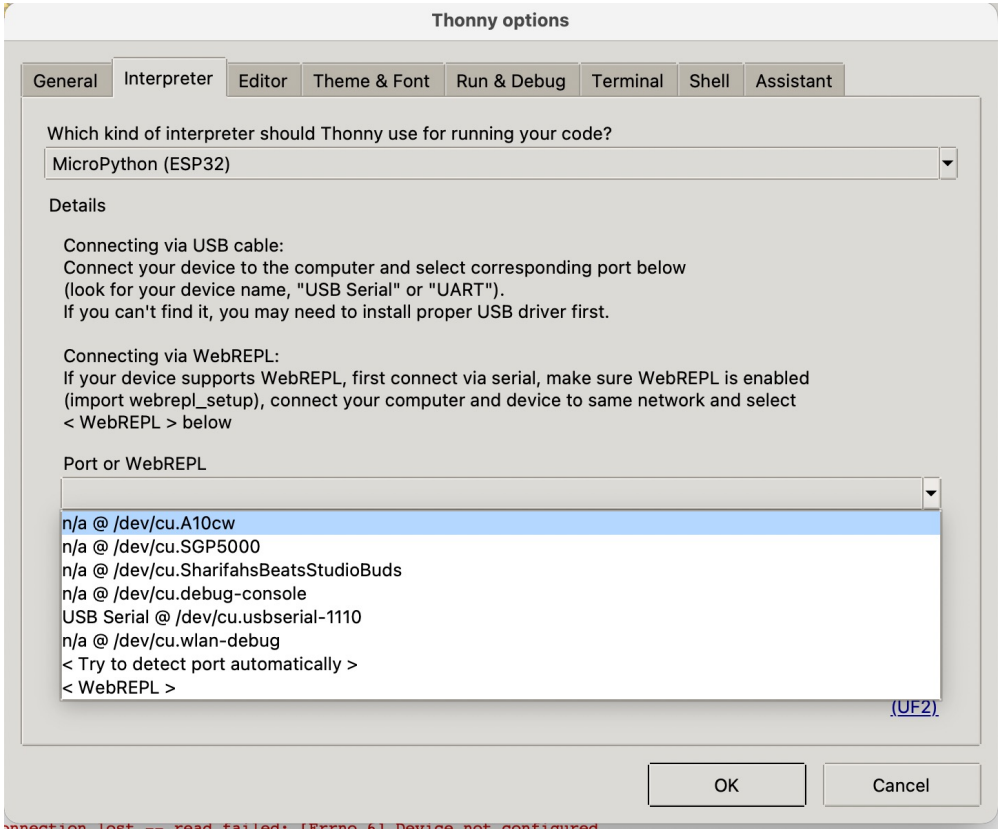
- ESP32 uses 3.3V logic
- The expansion board includes both 3.3 and 5.0V. Do not mixed them
- Wrong voltage can damage pins
- Never short 3.3V or 5.0V to GND
- Always power off before rewiring
- Check polarity (+ and -)

Defining COMM Port

- Evry time to change port or restart a RUN, be sure to go to tab run on the top left

->Configure interpreter-> (USB port)

- Click OK and run your code.
- Else if will not identify the communication port of the laptop/computer to the ESP32



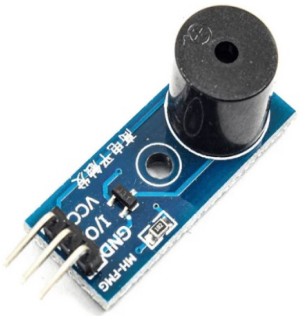
Smoke Sensor and Passive Buzzer

Smoke sensor MQ135

The MQ-135 is a low-cost semiconductor gas sensor module designed to detect harmful gases, including ammonia, nitrogen oxides, alcohol, benzene, smoke. Operating at +5V, it provides both analog and digital (TTL) outputs, featuring high sensitivity for air quality monitoring and a 20-second preheat time.

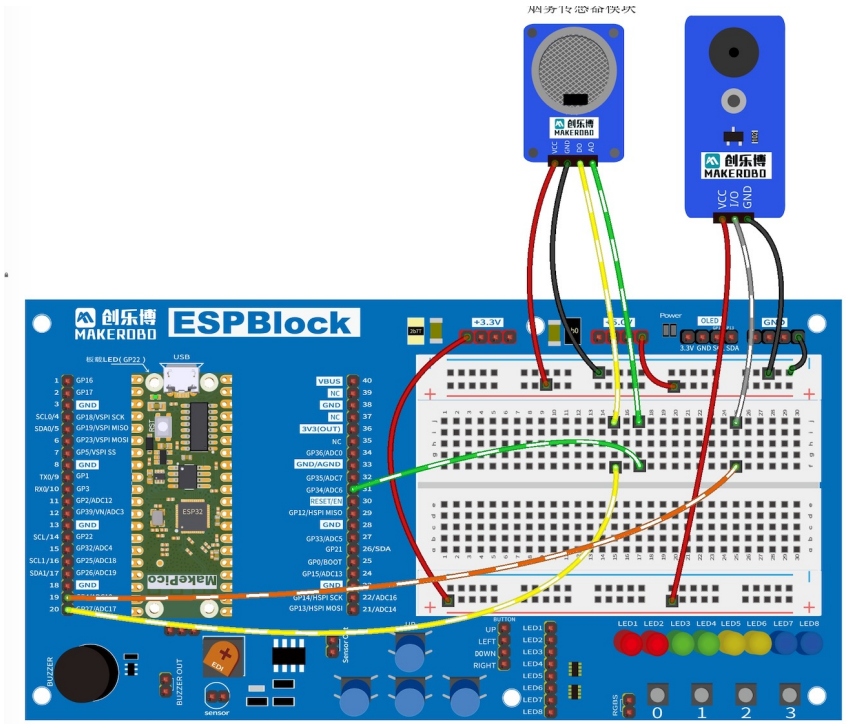
Passive Buzzer

An electronic sound-making component that requires an external alternating current (AC) or PWM signal to produce sound, lacking an internal oscillator. It can vary pitch, create musical notes, or produce complex tones by modulating the input frequency. Often operating at 3-5V.



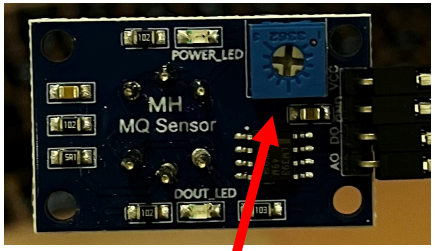
ESP32 to Smoke Sensor and Buzzer

- Make sure the power is OFF (disconnected)
- Connect the GPIO pins to the sensor and buzzer pins
- Check connections
- Connect the board to the laptop/ computer
- Open the gas_sensorV2.py code
- Define COMM port
- Click RUN



```
from machine import Pin, ADC, PWM
from time import sleep, ticks_ms

# Pin Setup
SMOKE_DO = 27      # Digital output from sensor
SMOKE_AO = 34      # Analog output from sensor
BUZZER_PIN = 4     # Passive buzzer

# Initialize
smoke_digital = Pin(SMOKE_DO, Pin.IN)
smoke_analog = ADC(Pin(SMOKE_AO))
smoke_analog.atten(ADC.ATTN_11DB)

buzzer = PWM(Pin(BUZZER_PIN))
buzzer.duty(0)     # OFF initially

# Functions
def buzzer_on():
  buzzer.freq(1000)    # 1 kHz tone
  buzzer.duty(512)     # 50% duty

def buzzer_off():
  buzzer.duty(0)

def print_safe(adc):
  print("SAFE | Gas Level =", adc)

def print_danger(adc):
  print("WARNING! SMOKE DETECTED | Gas Level =", adc)
```

```
def print_safe(adc):
  print("SAFE | Gas Level =", adc)

def print_danger(adc):
  print("WARNING! SMOKE DETECTED | Gas Level =", adc)

# Main Loop
last_print = 0
state = 1

while True:
  digital = smoke_digital.value()
  analog = smoke_analog.read()

  # Print every 1 sec
  if ticks_ms() - last_print > 1000:
    if digital == 1:
      print_safe(analog)
    else:
      print_danger(analog)
    last_print = ticks_ms()

  # Alarm
  if digital == 0:
    buzzer_on()
  else:
    buzzer_off()

  sleep(0.1)
```

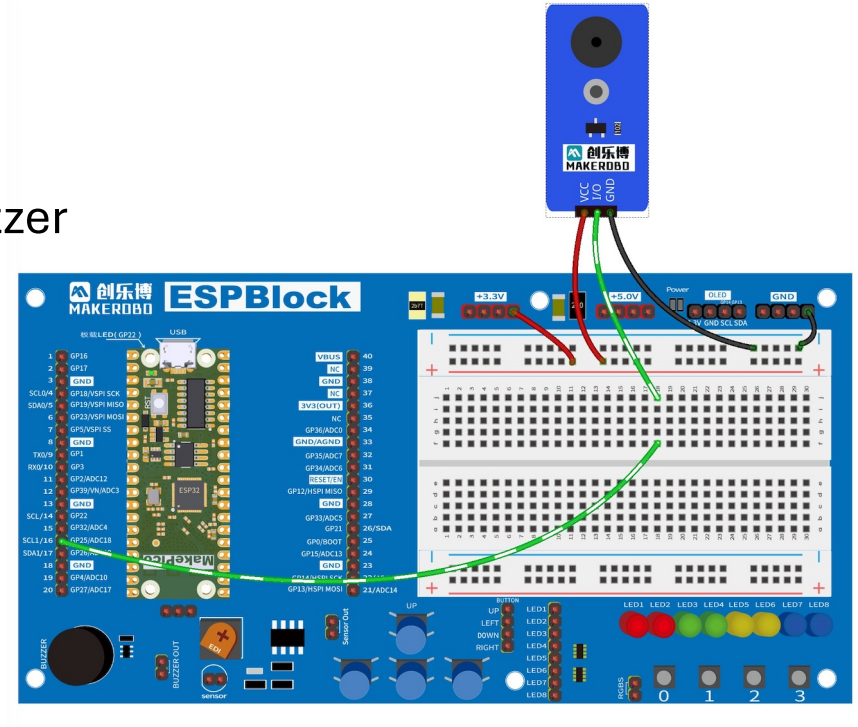


Tips: use the screwdriver to turn the potentiometer slowly while reading the DO

Gas_sensorV2.py code

Creating Song with passive buzzer

- Make sure the power is OFF (disconnected)
- Connect the GPIO pins to the buzzer pins
- Check connections
- Connect the board to the laptop/ computer
- Open the Passive_buzzer.py code
- Define COMM port
- Click RUN



Passive_buzzer.py code

```
from machine import Pin, PWM
from time import sleep

# Passive buzzer pin
BUZZER_PIN = 25

# Create PWM object
buzzer = PWM(Pin(BUZZER_PIN))
buzzer.duty_u16(0)   # Start silent

# Notes (Hz)
C4 = 262
D4 = 294
E4 = 330
F4 = 349
G4 = 392
A4 = 440
B4 = 494
C5 = 523

# Song: Twinkle Twinkle
melody = [
   C4, C4, G4, G4, A4, A4, G4,
   F4, F4, E4, E4, D4, D4, C4
]

beats = [
   0.4,0.4,0.4,0.4,0.4,0.4,0.8,
   0.4,0.4,0.4,0.4,0.4,0.4,0.8
]

# Play one tone
def play_tone(freq, duration):
   if freq == 0:
      buzzer.duty_u16(0)
   else:
      buzzer.freq(freq)
      buzzer.duty_u16(30000)   #
volume

   sleep(duration)

   # short pause between notes
   buzzer.duty_u16(0)
   sleep(0.05)

# Main loop
try:
   while True:
      for i in range(len(melody)):
         play_tone(melody[i], beats[i])
      sleep(1)

except KeyboardInterrupt:
   buzzer.deinit()
   print("Stopped")
```

Use different code for different outcome

ESP32-Passive_buzzerV2.py

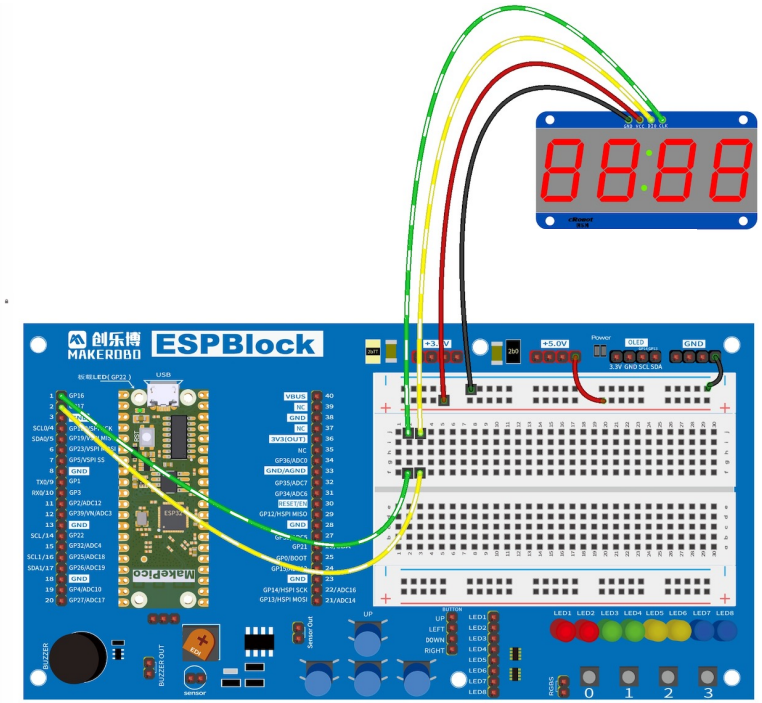
- Mary had a little lamb Song

7 Segment Display

- This is a popular, low-cost 4-digit, 7-segment LED display module used with microcontrollers to show numbers, time, and basic characters.
- It requires only two data pins (DIO and CLK) for control and supports adjustable brightness (0-7).
- It is commonly used for clocks, timers, and sensors.

ESP32 to TM1637 Digit Display connections

- Make sure the power is OFF (disconnected)
- Connect the GPIO pins to the display pins
- Check connections
- Connect ESP32 to the laptop/ computer
- Open the 7seg-display.py code
- Define COMM Port
- Click RUN



TM1637.py code

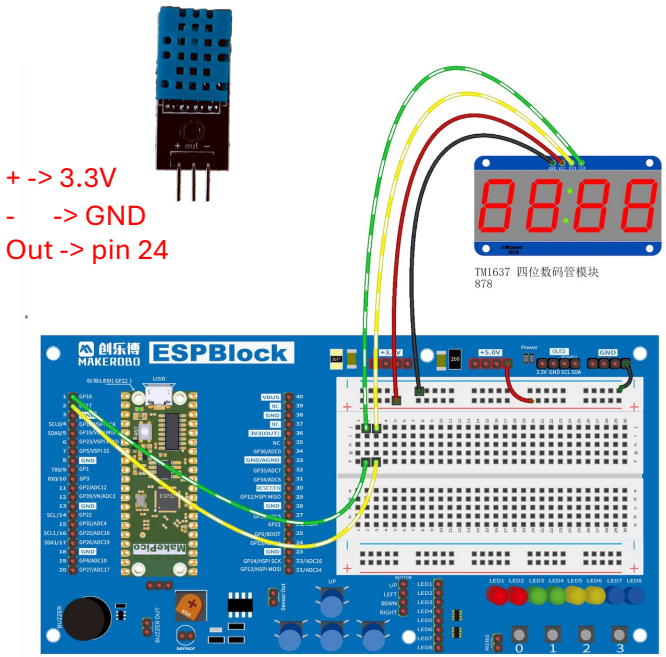
```
from micropython import const
from machine import Pin
from time import sleep, sleep_us

# TM1637 Commands
TM1637_CMD1 = const(64)
TM1637_CMD2 = const(192)
TM1637_CMD3 = const(128)
TM1637_DSP_ON = const(8)

# Digit segments 0-9
digits = bytearray(b'\x3f\x06\x5b\x4f\x66\x6d\x7d\x07\x7f\x6f')

# TM1637 Class
class TM1637:
  def __init__(self, clk, dio, brightness=7):
    self.clk = Pin(clk, Pin.OUT)
    self.dio = Pin(dio, Pin.OUT)
    self.brightness = brightness
    self.clk.value(0)
    self.dio.value(0)

  def start(self):
    self.dio.value(1)
    self.clk.value(1)
    self.dio.value(0)
    self.clk.value(0)

def stop(self):
    self.clk.value(0)
    self.dio.value(0)
    self.clk.value(1)
    self.dio.value(1)

  def write_byte(self, b):
    for i in range(8):
      self.clk.value(0)
      self.dio.value((b >> i) & 1)
      sleep_us(5)
      self.clk.value(1)
      sleep_us(5)

    self.clk.value(0)
    self.dio.value(1)
    self.clk.value(1)
    sleep_us(5)
    self.clk.value(0)

  def show(self, data):
    self.start()
    self.write_byte(TM1637_CMD1)
    self.stop()

    self.start()
    self.write_byte(TM1637_CMD2)

    for i in data:
      self.write_byte(i)

    self.stop()

    self.start()
    self.write_byte(TM1637_CMD3 | TM1637_DSP_ON | self.brightness)
    self.stop()

  def number(self, num):
    num = max(0, min(num, 9999))
    s = "{:0>4d}".format(num)

    data = []
    for c in s:
      data.append(digits[int(c)])

    self.show(data)

# MAIN PROGRAM
tm = TM1637(clk=16, dio=17)

n = 0

while True:
  tm.number(n)
  n += 1
  if n > 9999:
    n = 0
  sleep(1)
```

Use different code for different outcome

ESP32-TM1637-Clock.py

- Same connection

ESP32-TM1637-DHT.py



SW-420 Vibration Sensor

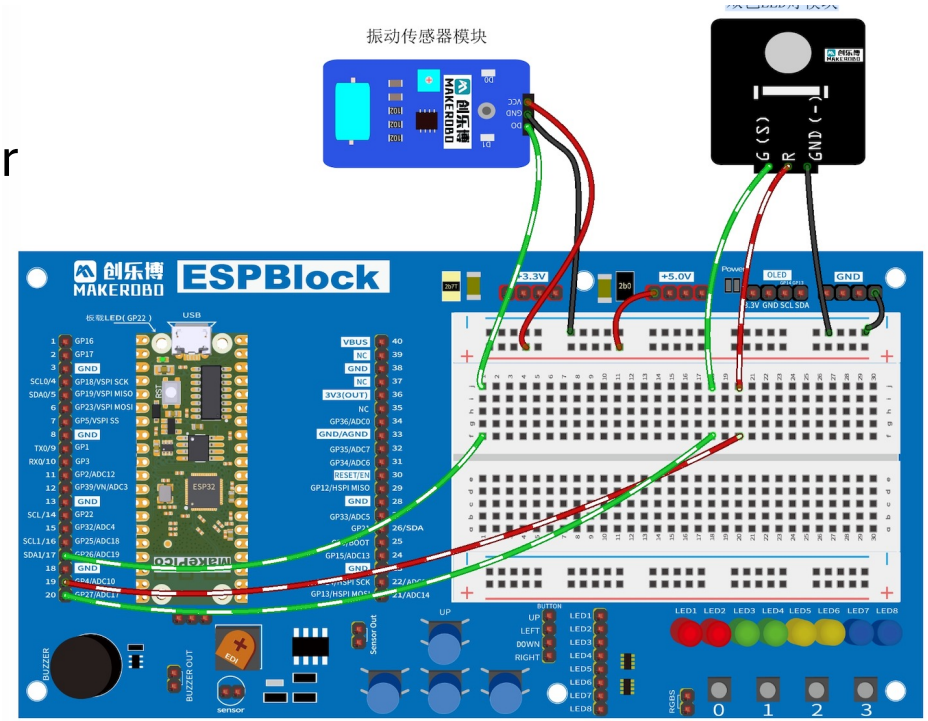
The SW-420 is a normally closed, non-directional vibration sensor module that detects motion or shock using an internal spring-loaded switch.

It operates between 3.3V-5V, utilizes an LM393 comparator for digital output (0/1), and features a potentiometer to adjust sensitivity for applications like alarms and motor testing

You can trigger this sensor by putting it on a washing machine (something that vibrates) or simply tapping on it.

ESP32 to SW-420 connection

- Make sure the power is OFF (disconnected)
- Connect the GPIO pins to the sensor pins
- Check connections
- Connect ESP32 to the laptop/ computer
- Open the vibration_sensor.py code
- Define COMM Port
- Click RUN



Vibrator_switch.py

```
from machine import Pin
from time import sleep_ms

sensor = Pin(26, Pin.IN)
red = Pin(4, Pin.OUT)
green = Pin(27, Pin.OUT)

while True:
  if sensor.value() == 0:
    print("SHAKE DETECTED")
    red.on()
    green.off()
    sleep_ms(300)
  else:
    red.off()
    green.on()

  sleep_ms(50)
```

Try and break the sensor circuit. Observe the outcome

There are a few set of codes in the github that you can try that will give you different outputs.